
SignMuon: fast as Muon, communication-effective as SignSGD

Maria Smirnova
MIPT
Moscow, Russia
smirnova.ma@phystech.edu

Alexey Kravatskiy
MIPT
Moscow, Russia
email@phystech.edu

Dmitry Kovalev
MIPT
Moscow, Russia
email@phystech.edu

Abstract

SignSGD is a popular algorithm due to its strong performance in both centralized and decentralized settings: it performs competitively with Adam while being communication-efficient, transmitting up to 32x fewer bits than standard optimizers. This work introduces SignMuon, an algorithm that combines the structured LMO-based update of Muon with the sign compression of SignSGD. Our experiments show that SignMuon achieves performance nearly on par with Muon in the centralized setting and outperforms SignSGD in both centralized and federated settings. We establish theoretical convergence guarantees for SignMuon in the smooth non-convex regime. We further analyze specific modifications to the algorithm that enhance numerical stability without compromising communication efficiency. Finally, this work extends our framework to the broader class of SignA algorithms, where A denotes any LMO-based optimizer, providing a unified convergence analysis for sign-compressed LMO methods. Empirical validation is conducted on synthetic convex and nonconvex problems with known smoothness constants, CIFAR-airbench, federated MNIST/CIFAR-10 classification, and NanoGPT training.

Keywords: TODO

1 Введение (Introduction)

Современные методы обучения глубоких нейронных сетей (DNN) требуют использования оптимизаторов, которые не только обеспечивают высокую скорость сходимости, но и минимизируют коммуникационные издержки в распределённых и федеративных системах, где передача обновлений между узлами является узким местом (bottleneck). Так, алгоритм SignSGD продемонстрировал, что даже экстремальная компрессия (до 1 бита на компоненту) сохраняет работоспособность метода и позволяет сократить объём передаваемых данных примерно в 32 раза при незначительной потере качества модели по сравнению с SGD [1, 2].

SignSGD не учитывает матричную структуру параметров: в задачах с матричными весами он рассматривает матрицы как одномерные векторы (flattening), игнорируя их внутреннюю 2D-тензорную структуру, что может ухудшать качество обучения DNN. Недавно предложенный оптимизатор Muon показал, что его ортонормированные обновления стабилизируют обучение DNN и в ряде случаев превосходят стандартные адаптивные методы оптимизации по качеству итогового решения [3–5]. Оптимизатор Muon для матричных параметров реализует шаг steepest descent в спектральной норме. Согласно Bernstein and Newhouse [3], Muon вычисляет направление обновления как ортонормированную матрицу $UV^\top$, где U и V – ведущие сингулярные векторы градиента.

Однако Muon требует передачи полных матриц обновлений, что в федеративных задачах может быть слишком затратным. В условиях федеративного обучения данные распределены между

узлами, которые обмениваются информацией с сервером с целью обучения общей модели. Каждый узел имеет локальный набор данных, а целью федеративного машинного обучения является минимизация усредненной функции потерь по всем узлам. Поскольку участники сети часто находятся на значительном расстоянии от агрегирующего сервера и могут иметь ограниченную пропускную способность канала связи, возникает коммуникационное узкое место. Для решения этой проблемы обычно используются методы компрессии градиентов и обновлений [1, 6–8].

В данной работе мы предлагаем алгоритм SignMuon, который объединяет методы обновления Muon с экстремальной компрессией SignSGD. В отличие от SignSGD, где передаётся знак стохастического градиента, наш метод сначала вычисляет направление спуска с помощью LMO-обновления, как оптимизатор Muon, а затем передаёт только знак этого направления. Такая схема позволяет учитывать матричную структуру параметров и сохранять преимущества Muon, одновременно сокращая объём передаваемой информации до одного бита на компоненту. Тем самым SignMuon стремится объединить быструю сходимость структурированных оптимизаторов с коммуникационной эффективностью знаковых методов.

Эффективность предложенного оптимизатора показана на различных синтетических и прикладных задачах:

- Синтетический эксперимент по методу наименьших квадратов;
- Бенчмарк CIFAR-10 airbench [9]
- Классификация изображений MNIST
- Предварительное обучение (pre-training) моделей NanoGPT и GPT-2 Medium на датасете FineWeb [10]

Наши эксперименты направлены на изучение того, насколько эффективно можно сочетать структурированные направления обновления Muon с экстремальной коммуникационной компрессией знаковых методов. В частности, мы анализируем, как передача только знака направления обновления влияет на качество и устойчивость обучения по сравнению с использованием полного Muon-обновления.

Экспериментальные результаты показывают, что использование знаковой компрессии для направлений Muon позволяет (to be continued...)

Таким образом, предложенный подход демонстрирует, что LMO-основанные методы могут быть успешно объединены с знаковой компрессией без существенной потери качества обучения. Более того, предложенная схема естественным образом обобщается на более широкий класс алгоритмов SignA, где A обозначает любой оптимизатор, использующий LMO-направления. Это открывает возможность построения новых коммуникационно-эффективных алгоритмов оптимизации.

2 Постановка задачи (Problem Statement)

Стохастическая оптимизация

Рассмотрим стохастическую задачу оптимизации для гладкой невыпуклой функции

$$\min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x}) = \mathbb{E}_{\xi \sim \mathcal{J}} [f(\mathbf{x}, \xi)], \quad (1)$$

где ξ – случайный вектор неизвестного распределения $\mathcal{J}$, $f(\mathbf{x}, \xi)$ – функция потерь модели (loss function). В контексте обучения DNN: $\mathbf{x} \in \mathbb{R}^d$ – вектор всех параметров, цель практической оптимизации – найти точку $\mathbf{x}$ с малой нормой градиента $\|\nabla f(\mathbf{x})\|$.

Федеративное обучение

Рассмотрим данную задачу в контексте федеративного обучения. Пусть данные распределены по M узлам, каждый узел m имеет локальную выборку D_m размера $|D_m|$ и соответствующую локальную функцию потерь $f_m(\mathbf{x}) = \mathbb{E}_{\xi \sim D_m} [f(\mathbf{x}, \xi)]$. Цель – минимизировать усредненную по всем узлам функцию:

$$F(\mathbf{x}) = \min_{\mathbf{x} \in \mathbb{R}^d} \frac{1}{M} \sum_{m=1}^M \mathbb{E}_{\xi_m \sim D_m} f(\mathbf{x}, \xi_m), \quad \min_{\mathbf{x} \in \mathbb{R}^d} F(\mathbf{x}). \quad (2)$$

В федеративном режиме узлы вычисляют локальные стохастические градиенты и периодически отправляют сообщения на сервер, канал коммуникации часто ограничен, поэтому критическим ресурсом является объём передаваемых данных [2, 6]. Некоторые практические приёмы в федеративном обучении и компрессии также рассмотрены в [7, 11, 12].

Возможные ограничения связаны с L -гладкостью функции f в неевклидовом пространстве, оснащённом нормой $\|\cdot\| : \|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\|_* \leq L\|\mathbf{x} - \mathbf{y}\|$, где $\|\cdot\|_*$ – дуальная норма, определяемую как $\|\mathbf{x}\|_* := \sup_{\|\mathbf{x}'\| \leq 1} \langle \mathbf{x}, \mathbf{x}' \rangle$ для всех $\mathbf{x} \in \mathbb{R}^d$ [13]. На практике возможно использовать ослабленные условия типа (L_0, L_1) [14]. Учитывая результаты существующих исследований, показавших неэффективность передачи полных 32-битных градиентов в распределённых системах, целесообразно рассмотреть сценарии с экстремальной компрессией, где целевой уровень сжатия составляет 1 бит на компоненту сообщения [1, 2, 7].

Linear Minimization Oracle (LMO)

Определим Linear Minimization Oracle (LMO) – оператор $A(\cdot)$ как решение задачи линейной минимизации на допустимом множестве:

$$A(\mathbf{G}) = \arg \min_{\mathbf{D} \in \mathcal{S}} \langle \mathbf{G}, \mathbf{D} \rangle. \quad (3)$$

где $\mathbf{G} \in \mathbb{R}^{m \times n}$ – градиент (или его стохастическая оценка), $\mathbf{D} \in \mathbb{R}^{m \times n}$ – направление обновления, $\langle \mathbf{A}, \mathbf{B} \rangle = \text{tr}(\mathbf{A}^\top \mathbf{B})$ – стандартное скалярное произведение матриц, а $\mathcal{S} \subset \mathbb{R}^{m \times n}$ – множество допустимых направлений [15]. В частности, ограничения можно задать через единичный шар некоторой нормы:

$$A(\mathbf{G}) = \arg \min_{\|\mathbf{D}\| \leq 1} \langle \mathbf{G}, \mathbf{D} \rangle. \quad (4)$$

Такой оператор выбирает направление наискорейшего спуска в заданной норме. Аналитическая и практическая роль LMO подробно обсуждается в работах по норм-ограниченным LMO-оптимизаторам [13, 15, 16].

Оптимизатор Muon

Оптимизатор Muon использует матричную структуру градиента. Muon реализует step steepest descent в спектральной норме, где направление обновления получается ортогонализацией матрицы градиента [3]. Пусть $\mathbf{G} = \mathbf{U}\Sigma\mathbf{V}^\top$ – сингулярное разложение матрицы $\mathbf{G} \in \mathbb{R}^{m \times n}$, где $\mathbf{U} \in \mathbb{R}^{m \times r}$, $\mathbf{V} \in \mathbb{R}^{n \times r}$ – ортонормированные матрицы сингулярных векторов, $\Sigma \in \mathbb{R}^{r \times r}$ – диагональная матрица сингулярных значений, $r = \text{rank}(\mathbf{G})$. Тогда LMO-направление имеет вид:

$$A(\mathbf{G}) = \mathbf{U}\mathbf{V}^\top \in \mathbb{R}^{m \times n}. \quad (5)$$

то есть выбирается ортонормированное направление, соответствующее решению задачи линейной минимизации. На практике Muon использует сглаженную оценку градиента с моментумом и его обновление выглядит следующим образом:

$$\begin{aligned} \mathbf{M}_t &= \mu \mathbf{M}_{t-1} + \mathbf{G}_t, \\ \mathbf{M}_t &= \mathbf{U}_t \Sigma_t \mathbf{V}_t^\top, \\ \mathbf{D}_t &= \mathbf{U}_t \mathbf{V}_t^\top, \\ \mathbf{X}_{t+1} &= \mathbf{X}_t - \eta_t \mathbf{D}_t. \end{aligned} \quad (6)$$

В исходной реализации Muon вычисление $\mathbf{U}_t \mathbf{V}_t^\top$ выполняется приближённо с помощью итераций Ньютона-Шульца и других полиномиальных схем, что позволяет эффективно ортогонализировать матрицу без явного вычисления SVD [17–19]. Интуитивно такая ортогонализация делает обновления изотропными и предотвращает доминирование нескольких направлений градиента, что особенно важно при обучении DNN [3, 5].

Класс алгоритмов SignA и SignMuon

Обозначим знаковую компрессию через поэлементную функцию $\text{sign}(\cdot)$, которая переводит каждый элемент матрицы (или вектора) в ± 1 . Определим класс алгоритмов SignA как:

$$\begin{aligned} A(\mathbf{G}_t) &: \mathbf{G}_t \mapsto \mathbf{D}_t, \\ \mathbf{s}_t &= \text{sign}(\mathbf{D}_t) \in \{\pm 1\}^{m \times n}, \\ \mathbf{X}_{t+1} &= \mathbf{X}_t - \eta_t \hat{\mathbf{s}}_t, \end{aligned} \quad (7)$$

где $\mathbf{G}_t, \mathbf{D}_t, \mathbf{X}_t \in \mathbb{R}^{m \times n}$, $\hat{\mathbf{s}}_t$ – агрегированное сообщение (в централизованном случае $\hat{\mathbf{s}}_t = \mathbf{s}_t$, в федеративном – majority-vote или усреднение по узлам). Частным случаем данного семейства алгоритмов является SignMuon, где оператор A соответствует Muon (т.е. $\mathbf{D}_t = \mathbf{U}_t \mathbf{V}_t^\top$), и передаётся только $\text{sign}(\mathbf{U}_t \mathbf{V}_t^\top)$ [20].

SignA реализует шаг steepest-descent с ограничением по норме и ЛМО-направлением:

$$\mathbf{X}_{t+1} = \mathbf{X}_t - \eta_t \text{sign} \left(\arg \min_{\|\mathbf{D}\| \leq 1} \langle \mathbf{G}_t, \mathbf{D} \rangle \right). \quad (8)$$

Мы поставили цель сконструировать коммуникационно-эффективный метод, который обеспечивает теоретические и эмпирические гарантии сходимости, сравнимые с Muon по качеству и с SignSGD – по коммуникации:

$$\min_{t \leq T} \mathbb{E} \|\nabla f(\mathbf{x}_t)\|^2 = \mathcal{O}(T^{-1/2}), \quad (9)$$

при использовании коммуникационного бюджета порядка d бит на шаг (sign-кодирование). Для анализа модификаций с error-feedback и квантованием см. [11, 21].

3 Theory

В работе рассматривается семейство алгоритмов SignA, где A обозначает любой оптимизатор, использующий ЛМО-направления. Все алгоритмы семейства построены по единой структуре, состоящей из трёх последовательных этапов: обработка моментума, вычисление ЛМО-направления и знаковая компрессия обновления.

В формальной постановке класс алгоритмов SignA на каждой итерации t выполняет следующую последовательность обновлений:

$$\begin{aligned} \mathbf{M}_t &= \mu \mathbf{M}_{t-1} + \mathbf{G}_t, \\ \tilde{\mathbf{M}}_t &= \mathbf{G}_t + \mu \mathbf{M}_t \quad (\text{Nesterov momentum – optional}), \\ \mathbf{D}_t &= A(\tilde{\mathbf{M}}_t), \\ \mathbf{s}_t &= \text{sign}(\mathbf{D}_t) \in \{\pm 1\}^{m \times n}, \\ \mathbf{X}_{t+1} &= \mathbf{X}_t - \eta_t \lambda_{\text{mult}} \mathbf{s}_t, \end{aligned} \quad (10)$$

где μ – коэффициент моментума, $\lambda_{\text{mult}} > 0$ – масштабирующий множитель (по умолчанию $\lambda_{\text{mult}} = 1$), $\mathbf{G}_t$ – стохастический градиент, а $\text{sign}(\cdot)$ применяется поэлементно к ЛМО-направлению $\mathbf{D}_t$.

Предлагаемый в работе метод SignMuon является частным случаем класса алгоритмов SignA, он объединяет ортогонализированные ЛМО-направления оптимизатора Muon с экстремальной знаковой компрессией SignSGD. Ниже описаны две основные реализации данного алгоритма – централизованная и федеративная.

3.1 Централизованный сценарий

В централизованной постановке алгоритм SignMuon применяется к матричным параметрам модели. Практическая реализация алгоритма учитывает тензорную структуру параметров нейронной сети. Процесс оптимизации разделен на два потока[9]:

1. Матричные параметры (веса сверточных и полносвязных слоев, $n_{\text{dim}} \geq 2$) обновляются с помощью ЛМО-оракула.
2. Векторные параметры (смещения и веса слоев пакетной нормализации, $n_{\text{dim}} = 1$) оптимизируются стандартным методом Adam.

Первоначально такой подход был предложен Jordan и другими [4] в эмпирическом описании. Первый результат о теоретической сходимости был получен Li и Hong, которые проанализировали гладкую невыпуклую постановку [18]. Так же данный подход описан в работе Рябинина [16].

Вычисление ортонормированного направления $\mathbf{D}_t \approx \mathbf{U}_t \mathbf{V}_t^\top$ для матричных параметров осуществляется без явного вычисления сингулярного разложения (SVD). Вместо этого применяется итерационный алгоритм Ньютона-Шульца. После получения LMO-направления $\mathbf{D}_t$, к нему применяется поэлементная функция $\text{sign}(\mathbf{D}_t)$, что сокращает объем передаваемой информации до 1 бита на компоненту. Логика обновления представлена в Алгоритме 1:

Algorithm 1 Обновление матричных параметров алгоритмом SignMuon

Require: Текущие веса $\mathbf{X}_t$, градиент $\mathbf{G}_t$, буфер момента $\mathbf{M}_{t-1}$, коэффициент момента μ , шаг обучения η_t , число итераций Newton-Schulz ns_steps

Ensure: Обновленные веса $\mathbf{X}_{t+1}$

- 1: $\mathbf{M}_t \leftarrow \mu \mathbf{M}_{t-1} + \mathbf{G}_t$ ▷ Сглаженный градиент (накопление момента)
- 2: $\tilde{\mathbf{M}}_t \leftarrow \mathbf{G}_t + \mu \mathbf{M}_t$ ▷ Шаг Нестерова для ускорения сходимости
- 3: $\mathbf{Y} \leftarrow \text{ReshapeTo2D}(\tilde{\mathbf{M}}_t)$ ▷ Приведение тензора к матрице $m \times n$
- 4: $\mathbf{Y} \leftarrow \mathbf{Y} / (\|\mathbf{Y}\|_F + \varepsilon)$ ▷ Нормализация по норме Фробениуса
- 5: **for** $k = 1$ to ns_steps **do** ▷ Итерации Ньютона-Шульца
- 6: $\mathbf{A} \leftarrow \mathbf{Y} \mathbf{Y}^\top$
- 7: $\mathbf{Y} \leftarrow 3.4445 \mathbf{Y} - 4.7750 \mathbf{A} \mathbf{Y} + 2.0315 \mathbf{A}^2 \mathbf{Y}$
- 8: **end for**
- 9: $\mathbf{D}_t \leftarrow \text{ReshapeToOriginal}(\mathbf{Y})$ ▷ Восстановление исходной размерности
- 10: $\mathbf{s}_t \leftarrow \text{sign}(\mathbf{D}_t)$ ▷ Знаковая компрессия
- 11: $\mathbf{X}_{t+1} \leftarrow \mathbf{X}_t - \eta_t \mathbf{s}_t$ ▷ Обновление параметров
- 12: **return** $\mathbf{X}_{t+1}$

В отличие от SignSGD, где знаковая компрессия применяется непосредственно к стохастическому градиенту, в данной алгоритме сначала строится структурированное направление обновления $\mathbf{D}_t$, соответствующее LMO-шагу Muon, и только затем происходит знаковая компрессия. Тем самым SignMuon сохраняет геометрию ортонормированных обновлений и снижает объем передаваемой информации до одного бита на компоненту.

3.2 Федеративный сценарий

В федеративном обучении данные распределены между M клиентами (узлами). Каждый клиент m имеет свой локальный набор данных D_m и вычисляет локальную функцию потерь $f_m(\mathbf{x})$. Цель обучения состоит в минимизации усреднённой по всем клиентам функции $F(\mathbf{x})$. Клиенты не обмениваются данными между собой, они передают на сервер сжатые сообщения, что делает коммуникацию узким местом (bottleneck).

В федеративном сценарии использования алгоритма SignMuon каждый клиент независимо вычисляет знаковое LMO-направление, а сервер выполняет агрегацию и обновление глобальной модели. Каждый клиент сначала копирует глобальную модель, после этого делает n_steps локальных шагов стохастической оптимизации и на каждом шаге вычисляет градиент по текущему батчу. Все локальные градиенты суммируются, после чего усредняются, данный процесс повторяется до предела количества раундов rounds:

$$\mathbf{G}_t^{(m)} = \frac{1}{n_{\text{steps}}} \sum_{i=1}^{n_{\text{steps}}} \nabla f(\mathbf{X}_t; \xi_{t,i}^{(m)}).$$

$$\mathbf{s}_t^{(m)} = \text{sign}(\mathbf{D}_t^{(m)}) = \text{sign}(A(\mathbf{G}_t^{(m)})) \in \{\pm 1\}^{m \times n},$$

где $t \in [0, \text{rounds} - 1]$, $A(\cdot)$ — оператор Muon (Newton-Schulz ортогонализация). Таким образом, каждый клиент передаёт на сервер лишь 1 бит на компонент матричного параметра.

На сервере поступившие от клиентов сообщения агрегируются с помощью majority-vote. Обозначим агрегированный знак через $\mathbf{s}_t^{\text{agg}}$, тогда он задаётся формулой:

$$\mathbf{s}_t^{\text{agg}} = \text{sign} \left(\sum_{m=1}^M \mathbf{s}_t^{(m)} \right). \quad (11)$$

Операция majority-vote гарантирует, что результирующий знак s_t^{agg} равен +1 или -1 в каждой компоненте и соответствует «мнению большинства» клиентов.

Далее сервер применяет momentum уже к агрегированному знаку:

$$\mathbf{M}_t = \mu \mathbf{M}_{t-1} + (1 - \mu) \mathbf{s}_t^{\text{agg}},$$

где $\mathbf{M}_t$ — буфер импульса на сервере. Обновление глобальной модели для матричных параметров (кроме последнего слоя) выполняется по правилу:

$$\mathbf{X}_{t+1} = \mathbf{X}_t - \eta_t \cdot \text{sign}(\mathbf{M}_t).$$

Отдельно следует отметить обработку последних (классификационных) слоёв модели. Эти параметры не обновляются SignMuon-правилом, а оптимизируются с помощью AdamW.

Полный алгоритм федеративного SignMuon приведён ниже:

Algorithm 2 Federated SignMuon

Require: Начальная модель $\mathbf{X}_0$, число клиентов M , число раундов rounds, число локальных шагов n_steps, шаг обучения η_t , коэффициент импульса μ

Ensure: Обновлённая глобальная модель $\mathbf{X}$

```

1:  $\mathbf{M}_0 \leftarrow 0$  ▷ Инициализация буфера импульса
2: for  $t = 0$  to rounds - 1 do
3:   for  $m = 1$  to  $M$  do ▷ Параллельно на каждом клиенте
4:      $\mathbf{X}_t^{(m)} \leftarrow \mathbf{X}_t$  ▷ Получение глобальной модели
5:
6:      $\mathbf{G}_t^{(m)} \leftarrow \frac{1}{n\_steps} \sum_{i=1}^{n\_steps} \nabla f(\mathbf{X}_t^{(m)}; \xi_{t,i}^{(m)})$  ▷ Усреднённый градиент
7:
8:      $\mathbf{Y}_t^{(m)} \leftarrow \text{ReshapeTo2D}(\mathbf{G}_t^{(m)})$  ▷ Приведение тензора к матрице  $m \times n$ 
9:
10:     $\mathbf{Y}_t^{(m)} \leftarrow \mathbf{Y}_t^{(m)} / (\|\mathbf{Y}_t^{(m)}\|_F + \varepsilon)$  ▷ Нормализация по норме Фробениуса
11:
12:    for  $k = 1$  to ns_steps do ▷ Итерации Ньютона-Шульца (Muon)
13:       $\mathbf{A}_t^{(m)} \leftarrow \mathbf{Y}_t^{(m)} (\mathbf{Y}_t^{(m)})^\top$ 
14:       $\mathbf{Y}_t^{(m)} \leftarrow 3.4445 \mathbf{Y}_t^{(m)} - 4.7750 \mathbf{A}_t^{(m)} \mathbf{Y}_t^{(m)} + 2.0315 (\mathbf{A}_t^{(m)})^2 \mathbf{Y}_t^{(m)}$ 
15:    end for
16:
17:     $\mathbf{D}_t^{(m)} \leftarrow \text{ReshapeToOriginal}(\mathbf{Y}_t^{(m)})$  ▷ Восстановление размерности
18:
19:     $\mathbf{s}_t^{(m)} \leftarrow \text{sign}(\mathbf{D}_t^{(m)})$  ▷ Знаковая компрессия
20:
21:    Отправить  $\mathbf{s}_t^{(m)}$  на сервер
22:  end for
23:
24:   $\mathbf{s}_t^{\text{agg}} \leftarrow \text{sign}\left(\sum_{m=1}^M \mathbf{s}_t^{(m)}\right)$  ▷ Majority-vote агрегация на сервере
25:
26:   $\mathbf{M}_t \leftarrow \mu \mathbf{M}_{t-1} + (1 - \mu) \mathbf{s}_t^{\text{agg}}$  ▷ Обновление импульса
27:
28:   $\mathbf{X}_{t+1} \leftarrow \mathbf{X}_t - \eta_t \cdot \text{sign}(\mathbf{M}_t)$  ▷ Обновление матричных параметров
29:
30:   $\mathbf{X}_{t+1}^{\text{last}} \leftarrow \text{AdamW}(\mathbf{X}_t^{\text{last}}, \mathbf{s}_t^{\text{agg}})$  ▷ Обновление последнего слоя
31: end for
32: return  $\mathbf{X}_{\text{rounds}}$ 

```

Таким образом, федеративный SignMuon сохраняет все структурные преимущества Muon, одновременно обеспечивая экстремально низкую коммуникационную нагрузку (только знаки ± 1 от каждого клиента).

3.3 Гипотезы исследования

Сформулируем две основные гипотезы для теоретического и экспериментального исследования алгоритма SignMuon. Пусть $\{\mathbf{x}_t\}_{t \geq 0}$ – последовательность итерационных шагов SignMuon, а $F(\mathbf{x})$ – целевая функция в постановке федеративного обучения (в централизованной постановке $F = f$).

Для анализа алгоритмов в обоих сценариях мы сделаем несколько предположений (Assumption).

Assumption 1. Функция $F(\mathbf{x})$ является L -гладкой относительно нормы $\|\cdot\|$, т.е. для любых $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ выполняется:

$$\|\nabla F(\mathbf{x}) - \nabla F(\mathbf{y})\|_* \leq L\|\mathbf{x} - \mathbf{y}\|, \quad (12)$$

где $\|\cdot\|$ — дуальная норма.

Assumption 2. Стохастический градиент удовлетворяет:

- Unbiased:

$$\mathbb{E}_\xi[\nabla f(\mathbf{x}; \xi)] = \nabla f(\mathbf{x}), \quad (13)$$

- Bounded variance:

$$\mathbb{E}_\xi[\|\nabla f(\mathbf{x}; \xi) - \nabla f(\mathbf{x})\|^2] \leq \sigma^2, \quad \sigma > 0 \quad (14)$$

Гипотеза 1 (Сходимость). При выполнении условий Assumption 1 и Assumption 2 алгоритм SignMuon сходится в гладком невыпуклом режиме. В частности, предполагается, что выполняется та же оценка скорости сходимости, что и для SignSGD:

$$\min_{0 \leq t \leq T} \mathbb{E}[\|\nabla F(\mathbf{x}_t)\|^2] = \mathcal{O}(T^{-1/2}). \quad (15)$$

Иными словами, знаковая компрессия LMO-направления не ухудшает стандартный порядок сходимости в гладком невыпуклом случае.

Гипотеза 2 (Сохранение преимуществ Muon). Переход от LMO-направления Muon $\mathbf{D}_t = \mathbf{U}_t \mathbf{V}_t^\top$ к его знаковой компрессии $\text{sign}(\mathbf{U}_t \mathbf{V}_t^\top)$ сохраняет основную геометрию ортонормированных обновлений. В результате SignMuon должен сохранять качество обучения, близкое к полному Muon, при существенно меньших коммуникационных затратах и превосходить SignSGD как в централизованном, так и в федеративном режимах.

4 Вычислительный эксперимент (Computational Experiment)

Целью эксперимента является эмпирическая валидация алгоритма SignMuon, как оптимизатора, объединяющего высокую скорость сходимости LMO-оптимизаторов с коммуникационной эффективностью знаковой компрессии. Само исследование состоит из нескольких частей:

- **Централизованное обучение (Centralized Performance):** Сравнение динамики сходимости и итоговой точности (Accuracy) алгоритма SignMuon с классическими оптимизаторами Adam, SGD, SignSGD и Muon при фиксированном количестве эпох. Одной из целей является демонстрация того, что SignMuon обеспечивает высокое качество классификации, сопоставимое с Muon, и при этом превосходит по точности классический знаковый метод SignSGD при идентичной коммуникации (1 бит на параметр).
- **Федеративное обучение (Federated learning):** Оценка алгоритма в распределенной среде, где передача полных 32-битных матриц градиентов является bottleneck. Цель данного этапа исследования показать, что сжатие ортогонализированных LMO-обновлений позволяет достичь качества полного Muon при снижении объема передаваемых данных.
- **Синтетическая неевклидова оптимизация (Synthetic Benchmarks):** Исследование сходимости расширенного семейства алгоритмов SignA на контролируемых задачах. Использование синтетических сред с заранее известными константами гладкости и числами обусловленности позволяет ... TODO

- **Эксперимент на масштабируемость и LLM:** Применение алгоритма SignMuon с целью предварительного обучения (pre-training) языковых моделей архитектуры Transformer. Валидация проводится на базе высокооптимизированного фреймворка Modded-NanoGPT, что позволяет сравнить SignMuon с Muon и AdamW. ... TODO

Наборы данных. Эксперименты проводятся на синтетических и реальных наборах данных:

- **CIFAR-10:** Стандартный датасет для задачи классификации изображений. Содержит 60 000 цветных изображений. Каждое изображение представлено в виде тензора размерности $3 \times 32 \times 32$, где первый индекс соответствует количеству цветовых каналов (RGB). Для централизованного обучения датасет делится на обучающую и тестовую выборки. В федеративном сценарии обучающая выборка делится между узлами.
- **MNIST:** Датасет для задачи классификации изображений, используемый в бенчмарках распределенного обучения. Содержит 70 000 черно-белых изображений рукописных цифр размером 28×28 пикселей (1 канал). Как и в случае с CIFAR-10, для централизованного обучения датасет делится на обучающую и тестовую выборки, а для федеративного сценария обучающая выборка делится между узлами.
- **Синтетические данные:** TODO
- **FineWeb (для NanoGPT):** Текстовый корпус для pre-training языковых моделей, используемый в бенчмарках серии modded-nanogpt.

Конфигурация алгоритма. Для экспериментов на MNIST и CIFAR-10 используется 4-слойная сверточная архитектура (CNN). Оптимизация матричных параметров производится исследуемыми алгоритмами SignMuon, Muon, SignSGD, SGD, Adam. Векторные параметры (смещения и веса слоев пакетной нормализации, $n_{dim} = 1$) оптимизируются стандартным методом Adam [9]. В качестве метода ортогонализации для Muon и SignMuon используется алгоритм Ньютона-Шульца. Темп обучения (learning rate) варьируется в зависимости от примененного алгоритма, используется Nesterov Momentum (0.9) и косинусный планировщик скорости обучения (CosineAnnealingLR). Обновление шага обучения η_t на эпохе t осуществляется по следующей формуле:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{t}{T_{\max}} \pi \right) \right), \quad (16)$$

где $\eta_{\max}$ — начальный темп обучения алгоритма, $\eta_{\min} = 0$ — минимальное значение (в нашей конфигурации), а $T_{\max}$ соответствует общему числу эпох обучения.

Метрики оценки качества. Для оценки качества работы алгоритмов мы используем две стандартные метрики: точность классификации (Accuracy) и кросс-энтропийную функцию потерь (Cross-Entropy Loss). Обе метрики вычисляются как на обучающей, так и на тестовой выборках.

Пусть N — общее число примеров в выборке (или в тестовом множестве всех клиентов в федеративном режиме), $\hat{y}_i \in \{1, \dots, C\}$ — предсказанный класс для i -го примера, $y_i \in \{1, \dots, C\}$ — истинный класс, а $\hat{p}_{i,c}$ — предсказанная моделью вероятность класса c для примера i .

- Точность классификации (в процентах): $\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{y}_i = y_i) \times 100\%$,
- Функция потерь кросс-энтропия: $\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{p}_{i,c})$, где $y_{i,c}$ — one-hot кодировка истинного класса.

Эти метрики позволяют сравнивать сходимость и финальное качество всех рассматриваемых алгоритмов (SignMuon, Muon, SignSGD, SGD, Adam) как в централизованном, так и в федеративном сценариях.

Ожидаемые таблицы и графики. После проведения исследований ожидаем получения следующих результатов: график сравнение сходимости, по оси X — количество эпох (epochs), Y — точность на тестовой выборке / значение функции потерь; таблица сравнения конечных результатов обучения; график сравнения коммуникационной эффективности в распределенном обучении.

5 Предварительные результаты (Preliminary Report)

Централизованный сценарий. В качестве базовой модели для централизованного обучения была использована сверточная нейронная сеть, представляющая собой отображение из пространства изображений в пространство вероятностей классов. Архитектура сети включает два сверточных блока с пакетной нормализацией (BatchNorm) и MLP-классификатор (multilayer perceptron). Основным исследуемым алгоритмом является SignMuon.

Был реализован программный пайплайн и проведен базовый вычислительный эксперимент по централизованному обучению на наборах данных MNIST и CIFAR-10. На датасете CIFAR-10 модель обучалась в течение 50 эпох с использованием CosineAnnealingLR. Для валидации предложенного алгоритма SignMuon проводилось сравнение с оптимизаторами Adam, SGD, SignSGD и Muon. На рисунках ниже представлены результаты данного эксперимента:

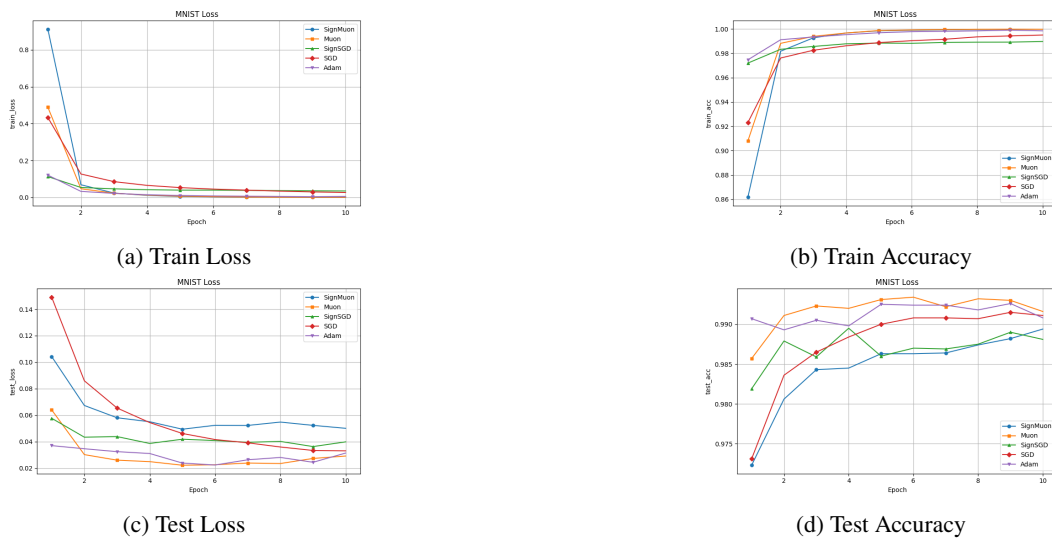


Рис. 1: Сравнение сходимости и точности различных алгоритмов оптимизации на обучении классификатора MNIST.

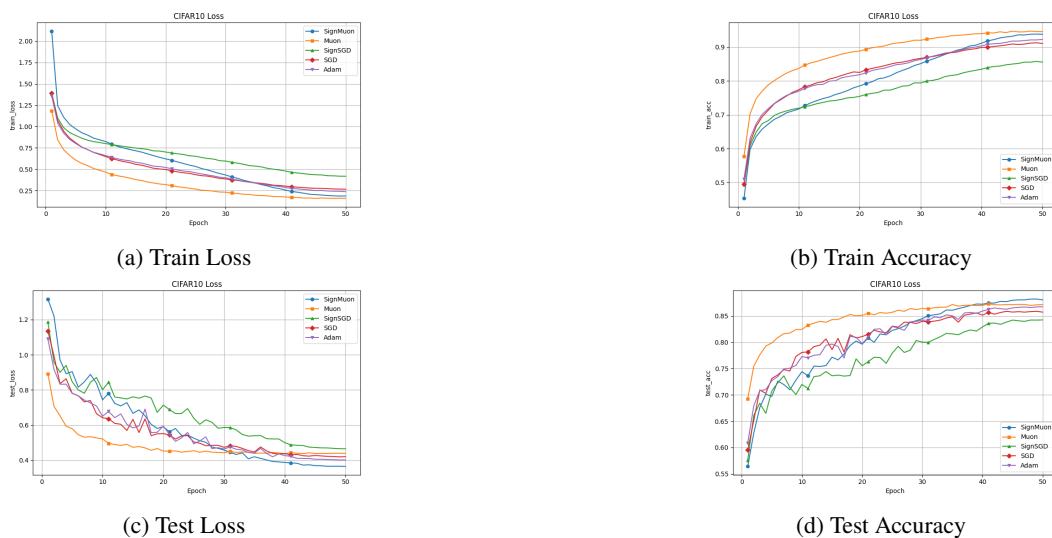


Рис. 2: Сравнение сходимости и точности различных алгоритмов оптимизации на обучении классификатора CIFAR-10.

Федеративный сценарий.

6 Discussion

TODO

7 Conclusion

TODO

Список литературы

- [1] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Anima Anandkumar. signsgd: Compressed optimisation for non-convex problems, 2018. URL <https://arxiv.org/abs/1802.04434>.
- [2] Jeremy Bernstein, Jiawei Zhao, Kamyar Azizzadenesheli, and Anima Anandkumar. signsgd with majority vote is communication efficient and fault tolerant, 2019. URL <https://arxiv.org/abs/1810.05291>.
- [3] Jeremy Bernstein and Laker Newhouse. Old optimizer, new norm: An anthology, 2024. URL <https://arxiv.org/abs/2409.20325>.
- [4] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- [5] Essential AI, :, Ishaan Shah, Anthony M. Polloreno, Karl Stratos, Philip Monk, Adarsh Chaluvaraju, Andrew Hojel, Andrew Ma, Anil Thomas, Ashish Tanwer, Darsh J Shah, Khoi Nguyen, Kurt Smith, Michael Callahan, Michael Pust, Mohit Parmar, Peter Rushton, Platon Mazarakis, Ritvik Kapila, Saurabh Srivastava, Somanshu Singla, Tim Romanski, Yash Vanjani, and Ashish Vaswani. Practical efficiency of muon for pretraining, 2025. URL <https://arxiv.org/abs/2505.02222>.
- [6] Samuel Horváth, Dmitry Kovalev, Konstantin Mishchenko, Sebastian Stich, and Peter Richtárik. Stochastic distributed learning with gradient quantization and variance reduction, 2019. URL <https://arxiv.org/abs/1904.05115>.
- [7] Aleksandr Beznosikov, Samuel Horváth, Peter Richtárik, and Mher Safaryan. On biased compression for distributed learning, 2024. URL <https://arxiv.org/abs/2002.12410>.
- [8] Dan Alistarh, Demjan Grubic, Jerry Li, Ryota Tomioka, and Milan Vojnovic. Qsgd: Communication-efficient sgd via gradient quantization and encoding, 2017. URL <https://arxiv.org/abs/1610.02132>.
- [9] K. Jordan. CIFAR-10 Airbench. <https://github.com/KellerJordan/cifar10-airbench>, 2023. Standardized benchmark for comparing optimizer performance on CIFAR-10 with standardized architecture.
- [10] K. Jordan et al. Modded-NanoGPT: Speedrunning the NanoGPT Baseline. <https://github.com/KellerJordan/modded-nanogpt>, 2024. Establishes a fast NanoGPT baseline for comparing optimizer performance in LLM training.
- [11] Kaja Gruntkowska, Alexander Gaponov, Zhirayr Tovmasyan, and Peter Richtárik. Error feedback for muon and friends, 2025. URL <https://arxiv.org/abs/2510.00643>.
- [12] Kwangjun Ahn, Byron Xu, Natalie Abreu, Ying Fan, Gagik Magakyan, Pratyusha Sharma, Zheng Zhan, and John Langford. Dion: Distributed orthonormalized updates, 2025. URL <https://arxiv.org/abs/2504.05295>.
- [13] D. Kovalev. Understanding Gradient Orthogonalization for Deep Learning via Non-Euclidean Trust-Region Optimization. 2025. URL <https://arxiv.org/abs/2503.12645>. Provides theoretical analysis of gradient orthogonalization via Non-Euclidean Trust-Region Optimization.

- [14] Thomas Pethick, Wanyun Xie, Mete Erdogan, Kimon Antonakopoulos, Antonio Silveti-Falls, and Volkan Cevher. Generalized gradient norm clipping non-euclidean (l_0, l_1) -smoothness, 2026. URL <https://arxiv.org/abs/2506.01913>.
- [15] Thomas Pethick, Wanyun Xie, Kimon Antonakopoulos, Zhenyu Zhu, Antonio Silveti-Falls, and Volkan Cevher. Training deep learning models with norm-constrained lmos, 2025. URL <https://arxiv.org/abs/2502.07529>.
- [16] Artem Riabinin, Egor Shulgin, Kaja Gruntkowska, and Peter Richtárik. Gluon: Making muon scion great again! (bridging theory and practice of lmo-based optimizers for llms), 2025. URL <https://arxiv.org/abs/2505.13416>.
- [17] Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. The polar express: Optimal matrix sign methods and their application to the muon algorithm, 2026. URL <https://arxiv.org/abs/2505.16932>.
- [18] Jiaxiang Li and Mingyi Hong. A note on the convergence of muon, 2025. URL <https://arxiv.org/abs/2502.02900>.
- [19] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for llm training, 2025. URL <https://arxiv.org/abs/2502.16982>.
- [20] Egor Petrov, Grigoriy Evseev, Aleksey Antonov, Andrey Veprikov, Nikolay Bushkov, Stanislav Moiseev, and Aleksandr Beznosikov. Leveraging coordinate momentum in signsgd and muon: Memory-optimized zero-order, 2025. URL <https://arxiv.org/abs/2506.04430>.
- [21] Aman Gupta, Rafael Celente, Abhishek Shivanna, D. T. Braithwaite, Gregory Dexter, Shao Tang, Hiroto Udagawa, Daniel Silva, Rohan Ramanath, and S. Sathiya Keerthi. Effective quantization of muon optimizer states, 2026. URL <https://arxiv.org/abs/2509.23106>.

A Appendix / supplemental material

A.1 Additional experiments / Proofs of Theorems

TODO